

JABATAN TENAGA MANUSIA
KEMENTERIAN SUMBER MANUSIA

PRODUCT REQUIREMENTS DOCUMENT

ICT Help Desk & Ticketing System (i-Aduan ICT)

A web-based ICT incident reporting, assignment and resolution-tracking platform

Built with: z.ai (GLM-5.2) | Backend: Supabase | Hosting: Netlify

Document Control

Revision History

Version	Date	Author	Description
0.1	14 Jul 2026	ICT Unit, JTM	Initial draft PRD created
1.0	14 Jul 2026	ICT Unit, JTM	Baseline PRD for development handoff to z.ai (GLM-5.2)

Approval

Role	Name	Email	Signature / Date
System Owner / Admin	En. A. Salleh Sabri	asallehsabri@jtm.gov.my	
Project Sponsor	Ketua Unit ICT, JTM	-	

Distribution List

- ICT Unit, Jabatan Tenaga Manusia
- System Administrator(s)
- Assigned Development Agent (z.ai / GLM-5.2)
- Pilot users: selected teachers and technicians

Table of Contents

1. Introduction

1.1 Purpose of this Document

This Product Requirements Document (PRD) defines the functional and non-functional requirements for the development of an ICT Help Desk and Ticketing System for Jabatan Tenaga Manusia (JTM). It is intended to guide an AI-assisted development workflow using z.ai (model GLM-5.2) as the code-generation agent, Supabase as the backend-as-a-service platform, and Netlify as the deployment/hosting platform. This document serves as the single source of truth for scope, features, data model, workflows, and acceptance criteria.

1.2 Background

JTM offices and training centres currently rely on ad-hoc channels (phone calls, WhatsApp, walk-ins) to report ICT-related problems such as WiFi/network outages, software issues, computer hardware faults, PA system malfunctions, and media equipment problems. This results in untracked requests, delayed resolution, no accountability trail, and no visibility into ICT unit workload. A centralised, role-based, auditable ticketing system is required to formalise this process.

1.3 Objectives

- Provide a single, authenticated channel for reporting ICT problems across defined categories.
- Enable transparent tracking of ticket status from issuance to resolution, with full timestamped history.
- Allow administrators to manage user accounts and assign tickets to the appropriate technician.
- Automate email notifications for assignment and resolution events, keeping all relevant parties informed.
- Provide a real-time, colour-coded dashboard visible to all authenticated users for at-a-glance status awareness.
- Establish a secure, low-cost, low-maintenance architecture suitable for a government department (Supabase + Netlify).

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
JTM	Jabatan Tenaga Manusia (Department of Manpower, Malaysia)
Ticket	A single ICT problem report submitted by a user
Issuer	The user (any role) who creates/reports a ticket
Assignee	The technician assigned to resolve a ticket
RLS	Row Level Security (Supabase/PostgreSQL access-control feature)
GLM-5.2	The z.ai large language model used as the AI coding agent for this build
SLA	Service Level Agreement / target resolution timeframe
PA System	Public Address system (audio announcement equipment)

2. Scope

2.1 In Scope

- Email/password authentication and role-based access control (Teacher, Technician, Admin).
- Forced password change on first login and email verification for all accounts.
- Ticket creation with category, summary, date and location.
- Admin-driven user management (create, invite, assign role, deactivate).
- Admin ticket assignment to technicians.
- Technician status updates with colour-coded workflow (Issued / In Progress / Resolved).
- Full audit trail of status changes with date/time and actor.
- Transactional email notifications (assignment, resolution) via a verified email service.
- Role-visible real-time dashboard with filters and summary statistics.
- Ticket status report / history view for all users.

2.2 Out of Scope (Phase 1)

- Mobile native applications (iOS/Android) — Phase 1 is responsive web only.
- SLA breach auto-escalation and penalty tracking.
- Integration with external asset/inventory management systems.
- Multi-language UI (Phase 1 ships in Bahasa Malaysia + English labels only).
- Ticket reopening / merging workflows.
- SMS notifications.

2.3 Assumptions

- All users hold a valid JTM-issued or JTM-registered email address.
- Admin accounts are provisioned manually by the ICT Unit; there is no public self-registration.
- Supabase free/pro tier and Netlify free/pro tier are acceptable for expected traffic volume.
- An outbound transactional email provider (e.g., Resend, SendGrid, or Supabase SMTP integration) will be connected to Supabase Edge Functions/Auth for all notification emails.

2.4 Constraints

- Government data-handling practices must be respected; passwords are never stored or transmitted in plain text beyond initial admin provisioning.
- The default administrator account (asallehsabri@jtm.gov.my) must be seeded during initial setup and its password rotated immediately after first login.

3. User Roles & Personas

3.1 Role Matrix

Capability	Teacher	Technician	Admin
Log in / manage own profile	Yes	Yes	Yes
Create a ticket	Yes	Yes	Yes
View own submitted tickets & status history	Yes	Yes	Yes

Capability	Teacher	Technician	Admin
View dashboard (all tickets, colour-coded)	Yes	Yes	Yes
Update ticket status (assigned tickets)	No	Yes	Yes
Assign ticket to technician	No	No	Yes
View all tickets (all issuers)	No	View assigned only	Yes
Add / manage user accounts & roles	No	No	Yes
Receive assignment notification email	N/A	Yes (as assignee)	No (unless assigned)
Receive resolution notification email	Yes (as issuer)	No	Yes

3.2 Personas

Cikgu Aminah (Teacher)

A training instructor who encounters a projector fault before class. She needs a fast, simple way to report the issue and check whether a technician has been assigned, without needing to call the ICT Unit.

En. Firdaus (Technician)

An ICT support officer who receives assigned tickets by email, updates status as he works (Issued → In Progress → Resolved), and needs visibility into his current workload via the dashboard.

En. A. Salleh Sabri (Administrator)

The ICT Unit head who onboards new staff accounts, triages incoming tickets, assigns them to the right technician, and monitors overall ICT service performance via the dashboard.

4. Functional Requirements

4.1 Authentication & Account Management

ID	Requirement	Priority
FR-1.1	Users log in using a registered email address and password; no public self-registration is permitted.	Must
FR-1.2	Admin can add a new user by entering name, email, and role (Teacher, Technician, or Admin). A temporary password is generated and the account is created in Supabase Auth.	Must
FR-1.3	On successful first login, the system detects a 'must_change_password' flag and forces the user to set a new password before proceeding.	Must
FR-1.4	New accounts must be verified via a confirmation link sent to the registered email	Must

ID	Requirement	Priority
	before the account becomes active.	
FR-1.5	Users can request a 'forgot password' reset link sent to their registered email.	Should
FR-1.6	Admin can deactivate/reactivate a user account without deleting historical ticket data.	Should
FR-1.7	Session tokens expire after a configurable period of inactivity and require re-authentication.	Should

4.2 Ticket Creation

ID	Requirement	Priority
FR-2.1	Any authenticated user (Teacher, Technician, Admin) can create a new ticket.	Must
FR-2.2	Ticket form captures: Category/Type (WiFi/Network, Software, Computer, PA System, Media), Summary of Issue (free text), Date (auto-filled, editable), and Location (free text or dropdown of predefined JTM locations).	Must
FR-2.3	System auto-generates a unique ticket reference number and records the issuer, creation timestamp, and initial status of 'Issued'.	Must
FR-2.4	Issuer receives on-screen confirmation and can immediately view the new ticket in 'My Tickets'.	Must
FR-2.5	Optional: allow attachment of a photo of the fault (e.g., broken equipment) to the ticket.	Could

4.3 Ticket Assignment (Admin)

ID	Requirement	Priority
FR-3.1	Admin can view all tickets across all issuers, filterable by category, status, location, and date range.	Must
FR-3.2	Admin can assign an unassigned ticket to a specific technician from a list of active Technician accounts.	Must
FR-3.3	Assigning a ticket records the assigning admin, the assigned technician, and a timestamp; status remains 'Issued' until the technician updates it.	Must
FR-3.4	Admin can re-assign a ticket to a different technician if required, with the change logged in the status history.	Should

4.4 Status Updates & Workflow

ID	Requirement	Priority
FR-4.1	Only the assigned technician (or an admin) can update the status of a given ticket.	Must
FR-4.2	Status values are restricted to: Issued (Red), In Progress (Yellow), Resolved (Green), applied in that logical order.	Must
FR-4.3	Each status change is appended to an immutable status-history log capturing: new status, actor (name/email), date and time.	Must

ID	Requirement	Priority
FR-4.4	The current status colour is reflected consistently across ticket list, ticket detail, and dashboard views.	Must
FR-4.5	Optional remarks/notes field for the technician to add context when changing status (e.g., 'Awaiting replacement part').	Should

4.5 Notifications (Email)

ID	Requirement	Priority
FR-5.1	When a ticket is assigned, the system sends an email to the assigned technician notifying them of the ticket details.	Must
FR-5.2	The same assignment email is CC'ed to the original issuer (teacher/user), informing them which technician is now handling their ticket.	Must
FR-5.3	When a ticket status changes to 'Resolved', the system sends an email notification to the issuer and to all Admin accounts.	Must
FR-5.4	All notification emails are sent via a transactional email service triggered from a Supabase Edge Function (or Database Webhook) — not from the client.	Must
FR-5.5	Failed email deliveries are logged for admin review and do not block the underlying status/assignment transaction.	Should

4.6 Reporting & Dashboard

ID	Requirement	Priority
FR-6.1	All authenticated users can view a dashboard summarising ticket counts by status, using the Red/Yellow/Green colour scheme.	Must
FR-6.2	Dashboard updates in near real-time as ticket statuses change (via Supabase Realtime subscriptions).	Must
FR-6.3	Users can view a full status report for any ticket they issued, showing the complete timestamped history.	Must
FR-6.4	Admins and Technicians can filter the dashboard by category, technician, location, and date range.	Should
FR-6.5	Dashboard displays summary charts (e.g., tickets by category, average resolution time) for Admins.	Could
FR-6.6	Users can export their own ticket history (CSV/PDF) — Admins can export all-ticket reports.	Could

5. Non-Functional Requirements

Category	Requirement
Security	All traffic served over HTTPS (enforced by Netlify). Passwords hashed via Supabase Auth (bcrypt). Row Level Security enforced on every table so users can only read/write data permitted by their role.

Category	Requirement
Security	Admin credentials (e.g., asallehsabri@jtm.gov.my) must have the seeded default password rotated immediately after first successful login; this is enforced by the same forced-password-change flow as all new users.
Availability	Target uptime of 99.5%, dependent on Supabase and Netlify SLAs.
Performance	Dashboard and ticket list views should load within 2 seconds under normal load (<100 concurrent users).
Usability	Mobile-responsive layout; ticket creation form completable in under 60 seconds on a phone browser.
Scalability	Data model supports growth to multiple JTM branches/locations without schema change.
Auditability	Every status change and assignment action is permanently logged with actor and timestamp; logs are not user-editable or deletable.
Maintainability	Codebase generated/maintained via z.ai (GLM-5.2) should follow a documented component structure and use environment variables for all Supabase/Netlify/email-provider secrets — no secrets committed to source control.
Compliance	Data handling aligned with Malaysian Personal Data Protection Act (PDPA) principles for staff personal data (name, email).
Backup	Daily automated database backups via Supabase's built-in backup feature.

6. System Architecture

6.1 High-Level Architecture

The system follows a Jamstack architecture: a static/SPA frontend hosted on Netlify communicates directly with Supabase (PostgreSQL database, Auth, Realtime, Storage, and Edge Functions) over HTTPS using the Supabase client SDK. Business logic that must run with elevated privilege (e.g., creating auth users as Admin, sending emails) executes in Supabase Edge Functions rather than in the browser.

- Client Layer: Web application (responsive), scaffolded and iterated using z.ai (model GLM-5.2) as the AI development agent.
- Backend-as-a-Service: Supabase — PostgreSQL database, Supabase Auth (email/password), Row Level Security, Realtime subscriptions, Storage (for optional attachments), and Edge Functions (serverless business logic).
- Notification Layer: Supabase Edge Function triggered by database webhook/trigger, calling a transactional email API (e.g., Resend or SendGrid) for assignment and resolution emails.
- Hosting/Deployment: Netlify — continuous deployment from Git repository, environment variables for Supabase URL/anon key, HTTPS by default, CDN-backed static asset delivery.

6.2 Recommended Technology Stack

Layer	Technology
AI Development Agent	z.ai — model GLM-5.2 (code generation, iteration, and refactoring)
Frontend Framework	React (or Next.js static export) with a component library (e.g., Tailwind CSS)

Layer	Technology
State/Data Fetching	Supabase JS client (@supabase/supabase-js), React Query for caching
Database	Supabase PostgreSQL
Authentication	Supabase Auth (email/password provider)
Authorisation	PostgreSQL Row Level Security (RLS) policies per role
Real-time Updates	Supabase Realtime (Postgres change subscriptions) for live dashboard
Serverless Functions	Supabase Edge Functions (Deno) for user provisioning and email triggers
Email Delivery	Transactional email API (e.g., Resend/SendGrid) invoked from an Edge Function
File Storage (optional)	Supabase Storage for ticket photo attachments
Hosting / CI-CD	Netlify (Git-based continuous deployment, environment variables, HTTPS)

6.3 Architecture Flow — Ticket Lifecycle

1. Teacher logs in and submits a ticket via the web app; a row is inserted into the tickets table with status = 'Issued'.
2. Admin views the unassigned ticket queue and assigns it to a Technician; the tickets row is updated with assigned_to and an entry is added to ticket_status_history.
3. A database trigger/webhook fires an Edge Function, which sends an assignment email to the technician and CCs the original issuer.
4. Technician updates status to 'In Progress' and later 'Resolved'; each change inserts a new ticket_status_history row and updates tickets.current_status.
5. On the 'Resolved' transition, an Edge Function sends a resolution email to the issuer and all Admin accounts.
6. All connected clients viewing the dashboard receive the updated status in real time via a Supabase Realtime subscription on the tickets table.

7. Data Model (Supabase / PostgreSQL)

All tables reside in the public schema and are protected by Row Level Security. auth.users (managed by Supabase Auth) holds login credentials; a linked profiles table holds application-specific role and status data.

7.1 profiles

Extends Supabase's built-in auth.users with application-level fields. One row per user, linked 1:1 via id.

Column	Type	Notes
id	uuid (PK, FK → auth.users.id)	Matches the Supabase Auth user id
full_name	text	Display name
email	text (unique)	Mirrors auth.users.email for convenience in queries
role	text (enum: teacher, technician, admin)	Determines RLS access and UI permissions

Column	Type	Notes
must_change_password	boolean, default true	Forces password change on first login
is_active	boolean, default true	Admin can deactivate an account
created_at	timestampz, default now()	Account creation time
created_by	uuid (FK → profiles.id)	Admin who created this account

7.2 categories

Lookup table for ticket types (seeded once; editable by Admin only).

Column	Type	Notes
id	smallint (PK)	Category identifier
name	text	'WiFi/Network', 'Software', 'Computer', 'PA System', 'Media'

7.3 tickets

Column	Type	Notes
id	uuid (PK, default gen_random_uuid())	Ticket identifier
ticket_no	text (unique)	Human-readable reference, e.g. JTM-2026-000123
category_id	smallint (FK → categories.id)	Type of ICT problem
summary	text	Description of the issue
location	text	Room/building/branch where the issue occurs
reported_date	date	Date of the reported issue (may differ from created_at)
issued_by	uuid (FK → profiles.id)	The user who created the ticket
assigned_to	uuid (FK → profiles.id), nullable	Technician assigned by Admin
assigned_by	uuid (FK → profiles.id), nullable	Admin who performed the assignment
current_status	text (enum: issued, in_progress, resolved)	Drives the Red/Yellow/Green indicator

Column	Type	Notes
created_at	timestampz, default now()	Ticket creation timestamp
updated_at	timestampz	Last modification timestamp (trigger-maintained)

7.4 ticket_status_history

Append-only audit log; one row is inserted for every status change or assignment event.

Column	Type	Notes
id	uuid (PK, default gen_random_uuid ())	History entry id
ticket_id	uuid (FK → tickets.id)	Related ticket
status	text	Status at this point in time (or 'assigned' as an event type)
remarks	text, nullable	Optional technician note
actor_id	uuid (FK → profiles.id)	User who made the change
changed_at	timestampz, default now()	Date and time of the update

7.5 notification_log

Column	Type	Notes
id	uuid (PK)	Log entry id
ticket_id	uuid (FK → tickets.id)	Related ticket
type	text (enum: assignment, resolution)	Which notification event
recipients	text[]	Email addresses the notification was sent to
status	text (enum: sent, failed)	Delivery outcome, for Admin troubleshooting
sent_at	timestampz, default now()	Timestamp of send attempt

7.6 Entity Relationship Summary

- profiles (1) — (many) tickets [via issued_by]
- profiles (1) — (many) tickets [via assigned_to]
- tickets (1) — (many) ticket_status_history
- tickets (1) — (many) notification_log

- categories (1) — (many) tickets

8. Ticket Status Workflow & Colour Coding

Every ticket progresses through a linear status workflow. The current status is always rendered with its associated colour across the ticket list, ticket detail page, and dashboard, giving all users an immediate visual read of ICT service health.

Status	Description
ISSUED	Default status when a ticket is created. Displayed in Red. Ticket is awaiting Admin triage/assignment to a technician.
IN PROGRESS	Displayed in Yellow/Amber. Set by the assigned Technician once work has begun on the reported issue.
RESOLVED	Displayed in Green. Set by the assigned Technician once the issue is fixed. Triggers resolution emails to the issuer and all Admins.

8.1 State Transition Rules

- A ticket is created only in the 'Issued' state; there is no 'unassigned/draft' pre-state.
- Only an Admin can move a ticket from unassigned to assigned (assignment does not itself change `current_status`, which remains 'Issued' until the technician acts).
- Only the assigned Technician or an Admin can transition status: Issued → In Progress → Resolved.
- Status cannot be set backward from Resolved without Admin override (e.g., if a fault recurs, a new ticket should be raised in Phase 1).
- Every transition is timestamped and attributed to the acting user in `ticket_status_history`.

9. Email Notification Specifications

9.1 Assignment Notification

Field	Value
Trigger	Admin assigns a ticket to a Technician (<code>tickets.assigned_to</code> is set)
To	Assigned Technician's registered email
CC	Issuer's (Teacher/user's) registered email
Subject (example)	[JTM ICT] Ticket JTM-2026-000123 Assigned to You — Computer Issue at Bilik Latihan 2
Body contents	Ticket number, category, summary, location, reported date, issuer name, and a direct link to the ticket in the system
Sender	Transactional email service, sent via Supabase Edge Function (not client-side)

9.2 Resolution Notification

Field	Value
Trigger	Technician (or Admin) sets ticket current_status to 'Resolved'
To	Issuer's registered email
CC	All active Admin accounts
Subject (example)	[JTM ICT] Ticket JTM-2026-000123 Resolved — WiFi/Network Issue at Makmal Komputer 1
Body contents	Ticket number, category, summary, resolving technician's name, resolution date/time, and a link to view the full status history
Sender	Transactional email service, sent via Supabase Edge Function (not client-side)

9.3 Account Provisioning Emails

- Welcome / Verify Email — sent when Admin creates a new account; contains a Supabase Auth email-confirmation link.
- Password Reset — sent on user-initiated 'forgot password' request.

10. Dashboard & Reporting Requirements

10.1 Dashboard Views by Role

Role	Dashboard Content
Teacher	Summary tiles (my Issued/In Progress/Resolved counts), list of tickets I raised with colour-coded status, and read-only view of all-ticket status counts.
Technician	My assigned tickets grouped by status colour, workload count, and read-only view of all-ticket status counts.
Admin	Full dashboard: all tickets by status/category/location/technician, unassigned-ticket queue, average resolution time, and user management access.

10.2 Ticket Status Report (Detail View)

Available to all roles for tickets they are permitted to view. Displays the ticket's core details (Type, Summary, Date, Location, Issuer), current colour-coded status, assigned technician (if any), and a chronological, timestamped history of every status change and assignment event.

10.3 Real-Time Behaviour

The dashboard subscribes to Supabase Realtime changes on the tickets and ticket_status_history tables so that status updates made by a technician are reflected for all viewing users within seconds, without requiring a page refresh.

11. Security & Access Control

11.1 Row Level Security Policy Summary

Table	Policy Summary
profiles	Users can SELECT their own row. Admins can SELECT/INSERT/UPDATE all rows. Users cannot change their own role.
tickets	Any authenticated user can INSERT. Users can SELECT rows where issued_by = auth.uid(). Technicians can additionally SELECT/UPDATE rows where assigned_to = auth.uid(). Admins can SELECT/UPDATE all rows.
ticket_status_history	Readable by any user who can read the parent ticket. INSERT restricted to the assigned technician or an Admin, matching the FR-4.1 rule.
categories	Readable by all authenticated users. Writable by Admins only.
notification_log	Readable and writable by Admins only (service role via Edge Function for inserts).

11.2 Additional Security Controls

- Admin-only operations that require elevated privilege (creating auth users, sending role-restricted invitations) are executed via a Supabase Edge Function using the service role key — never exposed to the browser.
- The Supabase anon key used by the frontend only ever grants access permitted by RLS policies.
- All secrets (Supabase service role key, email API key) are stored as Netlify/Supabase environment variables, never committed to the Git repository.
- Default seeded Admin password ('password123') must be rotated at first login via the mandatory password-change flow, and is documented here only as an initial provisioning value, not a permanent credential.

12. Deployment Plan

12.1 Environments

Environment	Purpose	Supabase Project	Netlify Site
Development	AI-assisted iteration with z.ai (GLM-5.2), local testing	supabase-jtm-ict-dev	Netlify deploy previews (per PR)
Staging/UAT	User acceptance testing with pilot teachers/technicians	supabase-jtm-ict-staging	jtm-ict-staging.netlify.app
Production	Live system for all JTM staff	supabase-jtm-ict-prod	jtm-ict.netlify.app (or custom domain)

12.2 Deployment Steps (Summary)

7. Provision Supabase project; run schema migrations (profiles, categories, tickets, ticket_status_history, notification_log) and enable RLS policies.

8. Seed categories table and the initial Admin account (asallehsabri@jtm.gov.my) with `must_change_password = true`.
9. Configure Supabase Auth email templates and connect the transactional email provider for Edge Function-triggered notifications.
10. Develop/generate the frontend application with z.ai (GLM-5.2), connecting to Supabase via environment-variable-configured client.
11. Connect the Git repository to Netlify; configure build command, publish directory, and environment variables (SUPABASE_URL, SUPABASE_ANON_KEY).
12. Deploy to Staging, conduct UAT with pilot users, then promote to Production.
13. Monitor notification_log and Supabase logs post-launch for delivery or access errors.

13. Acceptance Criteria (Sample)

AC-1: Ticket Creation

- Given a logged-in Teacher, when they submit the ticket form with Category, Summary, Date, and Location all completed, then a new ticket is created with status 'Issued' and appears in their 'My Tickets' list.

AC-2: First-Login Password Change

- Given a user logging in for the first time with an Admin-issued temporary password, when authentication succeeds, then the user is redirected to a mandatory 'Set New Password' screen before accessing any other page.

AC-3: Assignment Notification

- Given an Admin assigns Ticket JTM-2026-000123 to Technician En. Firdaus, when the assignment is saved, then an email is sent to En. Firdaus with the issuer's email in CC, within 60 seconds.

AC-4: Colour-Coded Status

- Given a ticket's current_status is 'in_progress', when any user views the dashboard or ticket list, then the ticket is displayed with a Yellow status indicator.

AC-5: Resolution Notification

- Given a Technician changes a ticket's status to 'Resolved', when the change is saved, then an email is sent to the issuer with all active Admins in CC, and a new ticket_status_history row is recorded with the technician's id and timestamp.

14. Risks & Mitigations

Risk	Impact	Mitigation
Email deliverability issues (spam filtering of gov domain)	Medium	Use a reputable transactional email provider with SPF/DKIM configured for the jtm.gov.my sending domain; log all attempts in notification_log.
Admin default password not rotated	High	Enforce forced password-change flow at code level; block access to any protected route until <code>must_change_password = false</code> .
Unauthorized data access across	High	Enforce PostgreSQL RLS on every table; do not rely on

Risk	Impact	Mitigation
roles		frontend-only checks.
Netlify/Supabase free-tier limits exceeded	Low	Monitor usage; budget for paid tier if concurrent users or storage grow beyond free-tier caps.
Technician updates status on ticket not assigned to them	Medium	RLS UPDATE policy on tickets restricts writes to assigned_to = auth.uid() OR admin role.

15. Development Milestones (Suggested)

Phase	Deliverable	Target Duration
Phase 1	Supabase schema, RLS policies, Auth setup, seeded Admin account	Week 1
Phase 2	Core frontend: login, forced password change, ticket creation, my-tickets view (built via z.ai GLM-5.2)	Week 2-3
Phase 3	Admin user management, ticket assignment, technician status updates	Week 3-4
Phase 4	Email notification Edge Functions (assignment, resolution)	Week 4-5
Phase 5	Real-time colour-coded dashboard and status report views	Week 5-6
Phase 6	UAT on Netlify staging, bug fixes, security review	Week 6-7
Phase 7	Production deployment and user onboarding	Week 8

16. Appendix

16.1 Sample RLS Policy (Pseudocode)

```
-- tickets: technicians may update only tickets assigned to them
```

```
CREATE POLICY technician_update_own_assigned ON tickets FOR UPDATE USING (assigned_to =
auth.uid() OR EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));
```

16.2 Sample Ticket JSON Shape

```
{ "ticket_no": "JTM-2026-000123", "category": "Computer", "summary": "Desktop PC does
not power on", "location": "Bilik Latihan 2", "reported_date": "2026-07-14",
"issued_by": "aminah@jtm.gov.my", "current_status": "issued" }
```

16.3 Category List

- WiFi/Network
- Software
- Computer
- PA System
- Media

16.4 References

- Supabase Documentation — <https://supabase.com/docs>
- Netlify Documentation — <https://docs.netlify.com>
- z.ai Platform — model GLM-5.2